

## SOFTWARE FOCUS

## OPEN ACCESS

# Automating Data Extraction From Scientific Literature and General PDF Files Using Large Language Models and KNIME: An Application in Toxicology

José Teófilo Moreira-Filho<sup>1</sup>  | Dhruv Ranganath<sup>1,2</sup> | Ricardo S. Tieghi<sup>1,2</sup> | Robert Patton<sup>3</sup> | Vicki Sutherland<sup>4</sup> | Charles Schmitt<sup>5</sup> | Andrew A. Rooney<sup>5</sup> | Jennifer Foster<sup>5</sup> | Vickie R. Walker<sup>5</sup> | Trey Saddler<sup>6</sup> | David Reif<sup>7</sup> | Kamel Mansouri<sup>1</sup> | Nicole Kleinstreuer<sup>1</sup>

<sup>1</sup>National Toxicology Program Interagency Center for the Evaluation of Alternative Toxicological Methods, Division of Translational Toxicology, National Institute of Environmental Health Sciences, Research Triangle Park, North Carolina, USA | <sup>2</sup>University of North Carolina, Chapel Hill, North Carolina, USA | <sup>3</sup>Oak Ridge National Laboratory, Oak Ridge, Tennessee, USA | <sup>4</sup>Preclinical Sciences & Translational Safety, Johnson & Johnson, Spring House, Pennsylvania, USA | <sup>5</sup>Division of Translational Toxicology, National Institute of Environmental Health Sciences, Research Triangle Park, North Carolina, USA | <sup>6</sup>Axle Informatics, Rockville, Maryland, USA | <sup>7</sup>Predictive Toxicology Branch, Division of Translational Toxicology, National Institute of Environmental Health Sciences, Research Triangle Park, North Carolina, USA

**Correspondence:** José Teófilo Moreira-Filho ([teofilo.moreirafilho@nih.gov](mailto:teofilo.moreirafilho@nih.gov)) | Kamel Mansouri ([kamel.mansouri@nih.gov](mailto:kamel.mansouri@nih.gov))

**Received:** 12 May 2025 | **Revised:** 15 August 2025 | **Accepted:** 25 August 2025

**Associate Editor:** Peter R. Schreiner | **Editor-in-Chief:** Peter R. Schreiner

**Funding:** This research was supported by the NIH, National Institute of Environmental Health Sciences through Intramural Research Program Project ES103376-02.

**Keywords:** generative artificial intelligence | KNIME | LLMs

## ABSTRACT

The large and steadily increasing volume of scientific publications presents a challenge in accessing and utilizing data due to their unstructured nature. Toxicology, in particular, depends on structured data from diverse study types for study evaluation, weight-of-evidence chemical assessments, and validation of new approach methodologies (NAMs). Manual data extraction is time and labor-intensive. This work presents an automated data extraction workflow using large language models (LLMs) within the KNIME platform. The workflow integrates document parsing tools with LLMs to extract variables from scientific publications and general PDF files. Two execution modes are available: text mode and image mode. Text mode applies tools for extracting text and tables, while image mode uses multimodal LLMs to process non-linear layouts and graphical content. The workflow achieves 81.14% accuracy in text mode for scientific publications and up to 98.54% in image mode for general PDF files. The KNIME platform ensures accessibility through a user-friendly interface, allowing non-experts to use advanced data extraction methods. This automated approach facilitates toxicological research by improving the retrieval of structured data. By democratizing access to LLM-powered workflows, this approach paves the way for significant advancements in knowledge synthesis to support biomedical research.

This article is categorized under:

Data Science > Artificial Intelligence/Machine Learning

Data Science > Computer Algorithms and Programming

Data Science > Databases and Expert Systems

This is an open access article under the terms of the [Creative Commons Attribution](https://creativecommons.org/licenses/by/4.0/) License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

© 2025 The Author(s). *WIREs Computational Molecular Science* published by Wiley Periodicals LLC.

## 1 | Introduction

An enormous volume of data is published in the scientific literature every year. According to bibliometric studies, over 2.5 million articles are published annually [1], and this number is increasing exponentially [2]. Such a volume of data is beyond the human capacity to find, process, read, and extract the relevant information, even within a single research domain. Consequently, invaluable knowledge often remains hidden within the sheer magnitude of scientific publications [3, 4]. Adding to the challenge, scientific findings are disseminated in various formats, including journal articles and reports, often presented in an unstructured manner reflecting the diverse ways researchers present and emphasize key findings via text, figures, tables, or schematics [5].

To overcome the challenges posed by the overwhelming volume and unstructured nature of scientific publications, one solution involves extracting data from free-text and organizing them into structured, searchable databases. These databases are computer-readable and enable researchers to query, process, and analyze data efficiently [3]. In toxicology, access to structured data is critical for identifying relevant information, evaluating study quality, integrating evidence on chemical effects, and developing reference datasets that can be used to build and validate new approach methodologies (NAMs), such as computational models. These NAMs address the challenge of informing hazard and risk assessment in a rapid, human-relevant way while reducing reliance on animal testing [6–9]. However, building tools and resources to make toxicology data accessible and leverage the information from all of this research requires significant effort in manual data extraction and curation, further complicated by the inherent difficulty and uncertainty in interpreting scientific information from diverse textual formats [10, 11].

In this context, Natural Language Processing (NLP), a subfield of artificial intelligence (AI) and linguistics that enables machines to process human words and phrases with their intent and content, has opened promising avenues for automating data extraction from scientific literature into structured datasets [12]. Despite its increase in popularity, traditional NLP methods often require considerable target-domain knowledge and the development of sets of domain-specific rules defined manually from experience on a case-by-case basis, making them time-intensive to fine-tune and with weak generalization capabilities [3, 13]. Fortunately, with the advent of large language models (LLMs), a new technological era has begun in the NLP field [14]. These models are trained using extensive amounts of data to achieve a level of linguistic comprehension akin to human understanding, and they can solve tasks for which they have not been explicitly trained [15, 16]. For a few examples, LLMs can answer questions, summarize text, change file formats, learn to use programs, execute multi-step tasks [17], and extract structured data from unstructured text [18–21].

Large language models can be accessed through provider-hosted web interfaces, which offer a straightforward way to interact with these powerful models. However, web interfaces often lack the flexibility and scalability needed for

more complex or automated applications. To overcome these limitations, LLMs can also be accessed through Application Programming Interfaces (APIs), enabling the development of automated, customizable, and scalable systems tailored to specific needs [22, 23]. These APIs allow for seamless integration of LLM capabilities into workflows, supporting tasks such as data extraction, information synthesis, and decision-making. Despite the advantages of APIs, relying solely on programming scripts for their implementation can pose accessibility challenges, particularly for non-technical users. This reliance on coding expertise can hinder the broader adoption and democratization of LLM-based technologies. To address this gap, the Konstanz Information Miner (KNIME), a free and open-source data analytics platform, provides a valuable alternative [24, 25]. KNIME's low-code/no-code environment, coupled with intuitive graphical user interfaces (GUIs), empowers users to rapidly design, deploy, and apply workflows without extensive programming knowledge [12]. By bridging the gap between technical complexity and user accessibility, KNIME facilitates the widespread adoption of LLM-powered systems, fostering innovation and inclusivity across diverse domains.

In this work, we introduce an automated KNIME workflow that easily integrates document parsing tools and LLMs to enable the efficient extraction of key study protocol and data variables relevant to toxicology. We also document the process used to establish and test this workflow for transparency and to support greater understanding of the rigor in the approach. The workflow is designed to extract data from Portable Document Format (PDF) files from various sources, including scientific publications and safety data sheets (SDSs). Multiple preprocessing strategies and extraction modes were explored to handle different document structures and improve accuracy. By leveraging the capabilities of LLMs such as GPT-4o and Claude 3.5 Sonnet within an accessible and user-friendly platform, this approach aims to streamline data extraction, enhance data accessibility, and support advancements in toxicological research and safety assessment. The performance testing demonstrated the workflow's reliability and efficacy, attaining an accuracy of 81.14% for scientific publications and up to 98.54% for SDSs. This robust assessment underscores the workflow's potential to democratize advanced data extraction methods, fostering broader adoption and interdisciplinary collaboration.

The data extraction workflow is part of the Modeling and Visualization Pipeline (MoVIZ), which has the goal of democratizing computational methods to non-experts and simplifying their use for the scientific community by providing user-friendly GUIs, step-by-step instructions, and straightforward tool installation [26]. This workflow is available for download from GitHub ([https://github.com/NIEHS/Data\\_extraction\\_workflow](https://github.com/NIEHS/Data_extraction_workflow)) and KNIME Community Hub (<https://hub.knime.com/s/oh8vLkMh-2h0th6B>) and can be installed on local desktops or network servers. Also, the workflow is accessible via the National Institute of Environmental Health Sciences (NIEHS) KNIME Server, serving all National Institutes of Health (NIH) users (under the NIH network) as a web application at the KNIME WebPortal (<https://knime.niehs.nih.gov/knime/webportal/>).

## 2 | Materials and Methods

### 2.1 | Background

The PDF file format first appeared in the early 1990s to enable consistent presentation of documents for viewing, sharing, and printing; the format quickly gained widespread acceptance in both academic and commercial fields. By 2015, Adobe estimated that approximately 1.5 trillion PDF files had been produced, which has undoubtedly increased exponentially in the years since [27]. PDF is a layout-oriented format that defines the placement and font of each individual character in the text, and the position of figures within the document [28, 29], and as a result, the semantic role of individual components is lost in this visually oriented file format. Without visually interpreting the pages, it becomes extremely challenging to determine whether a specific set of characters corresponds to a title, abstract, table, or another document element. Furthermore, scientific literature and reports in PDF format lack layout standardization across domains and publishers. These characteristics make PDF files particularly difficult to convert into structured, machine-readable formats [27, 29, 30].

In PDF data extraction tasks, identifying the document layout and extracting content while preserving semantic relationships is only the initial step. Subsequently, it is necessary to perform named entity recognition (NER) to obtain the information in a structured format, such as tabular data. NER is a subfield of NLP that focuses on identifying and classifying named entities (e.g., chemical names, animal species, administered doses, necropsy findings, etc.) within unstructured text [31]. LLMs have been proven effective for performing NER [20, 32, 33] due to their reasoning, comprehension, recognition capabilities, and ability to process large inputs with extensive context windows. Moreover, LLMs allow interaction through prompts—tailored instructions—making them more accessible, flexible, and suitable for human operators [13].

### 2.2 | Overview of the Data Extraction Workflow

Several approaches, machine learning models, and tools are available to automate PDF parsing, text extraction, and NER. However, no single tool suits all purposes, and it is essential to acknowledge that all existing tools have some drawbacks, either in terms of performance or ease of use. Furthermore, some of these tools are implemented in commercial products, which can hinder the democratization of these technologies [23, 27, 30, 34–36]. In this context, we developed a free and open-source KNIME workflow that combines various PDF parsing, text extraction, and NER approaches, integrated with LLMs, to produce structured output extractions. A general overview of the workflow steps is provided in Figure 1.

The data extraction KNIME workflow provides two different running modes: “New Analysis” and “New Analysis with Prior Configuration”. The first step of the default mode “New Analysis” is inputting a compressed archive file (ZIP format) containing PDF documents from scientific literature and other sources. These files are decompressed, and the individual PDFs are processed for parsing. The parsing process is divided into

two primary modes: text mode and image mode, designed to optimize data extraction while addressing the inherent challenges posed by the diverse structures and formats of PDF documents.

The text mode is subdivided into two categories: PDF files of scientific publications and general PDF files, which enable the workflow to extract textual data from PDF documents. For PDF files of scientific publications, the workflow extracts the text of documents that follow the IMRAD structure (Introduction, Methods, Results, and Discussion). The extracted elements typically include titles and main body sections, ensuring the document's logical flow and essential components are retained. For general PDF files, which may not necessarily follow the IMRAD format or require OCR (Optical Character Recognition) processing, the workflow ensures the extraction of the entire text content of the document. This approach is designed for diverse document structures and formats where no predefined headings or layouts are available.

The image mode is employed when documents contain complex layouts or significant non-textual elements such as images and graphs. This mode involves converting PDF pages into images, followed by preprocessing steps to standardize and enhance these images for further analysis. The prepared images are used as input for multimodal large language models (MLLMs), i.e., LLMs with abilities in natural language and visual information (image) modalities [37, 38], to extract textual and visual information. This mode is particularly effective for documents where traditional text extraction methods are insufficient due to the complexity of the layout or the visually graphical nature of the content.

Following the initial parsing and preprocessing of documents through text and image modes, LLMs and prompt engineering are applied for data extraction. In the context of textual data obtained from text mode, LLMs analyze the text to detect and classify words or sentences into categories relevant to toxicological research, such as chemical identifiers, dosages, and testing results. By leveraging their extensive pre-trained knowledge and contextual understanding, LLMs can discern nuances in language, differentiate between homonyms based on context, and identify entities that are not explicitly stated but are implied through complex linguistic structures. For data extracted through image mode, MLLMs are used to process and interpret the full multimodal context of PDFs, including textual content, layout, formatting, and graphical elements. The models are prompted to output a JavaScript Object Notation (JSON) file, which can be directly downloaded or converted at the end of the workflow to a comma-separated file (CSV) and/or Excel file (XLSX).

After a workflow finishes, the system captures the chosen settings and prompts as “flow variables”, which can be exported to a configuration file. That file can be used with the “New Analysis with Prior Configuration” mode to reproduce settings for new input files, ensuring consistent and efficient runs.

### 2.3 | Workflow Input and PDF Parsing

The data extraction KNIME workflow starts by the selection of one of two different running modes: “New Analysis” or “New

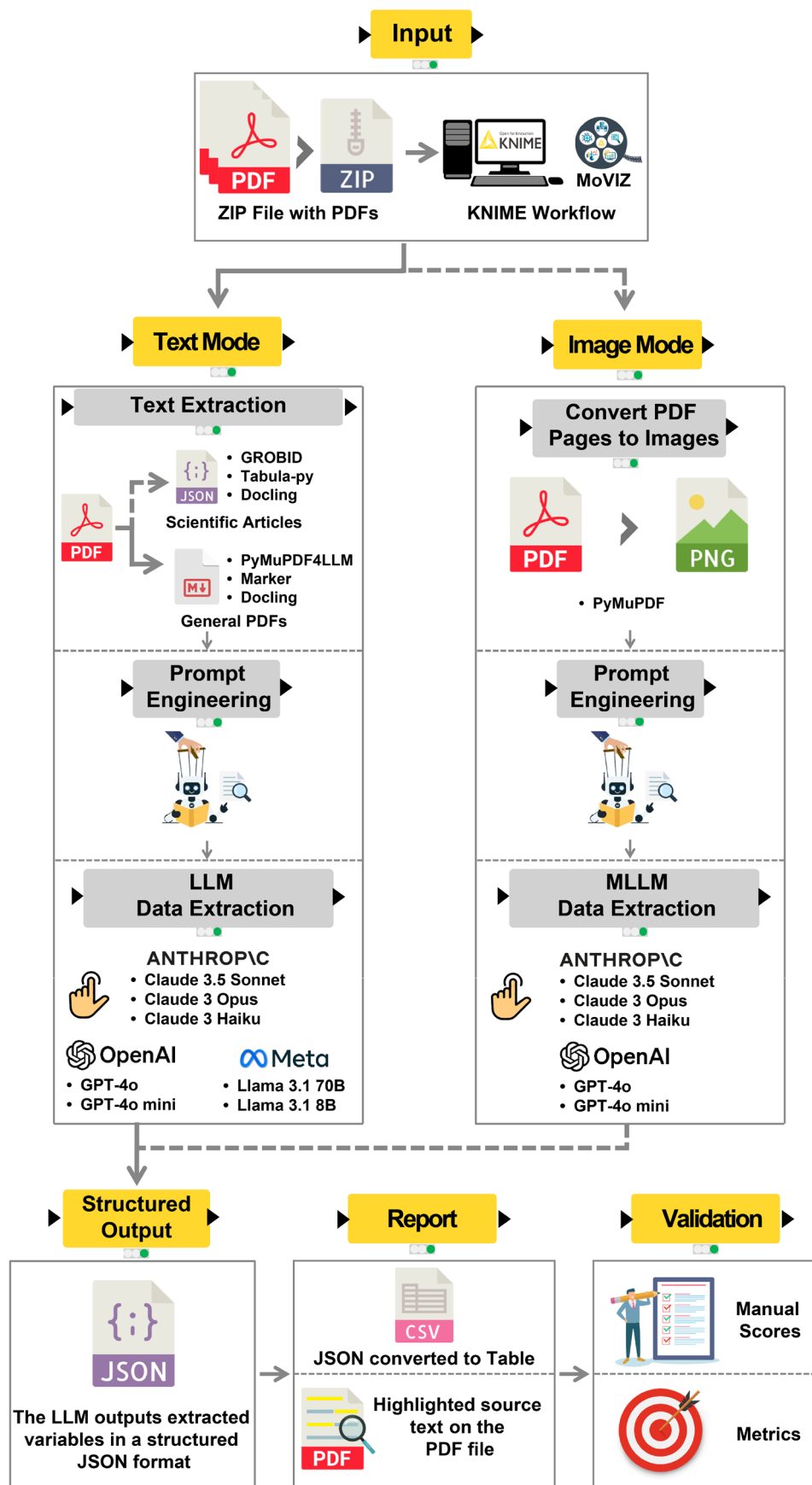


FIGURE 1 | Overview of the data extraction workflow.



Analysis with Prior Configuration” and with the input of a ZIP file containing the scientific literature or general PDF files for data extraction. The ZIP file serves as a centralized repository, enabling efficient handling of multiple documents in a single workflow execution. Each PDF file within the archive is expected to represent an individual document, such as a scientific article, regulatory report, or other text resource. The workflow extracts the ZIP file’s contents into a designated directory, and then the PDF files are parsed using the methods available in the text or image mode. The parsing mode is selected by the user based on the document type and extraction task; it is not an automated decision made by the tool.

After inputting PDF files, the text extraction process converts the documents into a machine-readable format. In the text mode for scientific articles, the GROBID [39] tool is employed to extract the complete text, including section titles and their corresponding content. The output consists of TEI (Text Encoding Initiative) XML files, parsed using a custom-built parser implemented with the ElementTree Python library [40]. The parser specifically processes elements within the <body> tag of the TEI XML structure, capturing the main sections of the document, such as the title, abstract, introduction, methods, results, and conclusions. For publications lacking a formal IMRAD structure (e.g., letters or communications), the entire body text is extracted as a single section. The output does not include sections such as references, author lists, and Supporting Information to reduce token usage. Finally, the parsed XML data is converted into a structured JSON format containing the document title and a list of sections, each represented by a title and its associated text content. In addition to the main body text of the scientific publications, specialized tools are needed for extracting tables. Tabular data in PDF files can be structured in different formats, oriented horizontally or vertically, and can vary from simple flat structures to complex hierarchical layouts, presenting unique challenges to table extraction [35, 41, 42]. Here, we tested different tools for extracting tabular data from PDF files: pdfplumber [43], tabula-py [44], Nougat [45], Marker [46], Docling [47], and Camelot [48]. After our tests, we selected tabula-py and Docling for our final implementation (details are in the [Supporting Information](#): 1. Details on the evaluation of tabular data extraction tools). All the tables in an article were extracted, converted to JSON, and combined with the GROBID output JSON file.

In the text mode, we implemented a second option for general PDF files that do not necessarily follow the IMRAD format or need OCR. In these cases, the extraction of the whole text content of the file is needed. The Python libraries Marker [46], Docling [47], or PyMuPDF4LLM [49] were used in this option to automate the parsing of PDF files into Markdown format, facilitating further data extraction. Marker [46] is a pipeline of deep learning models that extracts textual data from the PDF files and performs OCR using the Surya toolkit [50]. Using the Surya toolkit, Marker identifies and classifies various layout elements within the document, including titles, captions, tables, figures, textual blocks, and column and text order. Identifying these components is vital for accurately structuring the Markdown output and preserving the document’s original layout and semantic structure. In the final stage, Marker employs a model for post-processing the text, ensuring that the Markdown output is

clean and maintains the original document’s intent and accuracy [51]. Docling [47] uses PDF backends based on qpdf [52] or pypdfium [53] to extract text tokens such as string content and its coordinates on the page, as well as rendering bitmap images of each page. Docling then employs two AI models to enhance the document’s structure recognition. The first, a layout analysis model, uses advanced object detection to classify page elements from page images, such as text blocks and tables. The second, TableFormer, is adept at analyzing and reconstructing the structure of tables, even those with complex layouts. Optional OCR capabilities are also available through EasyOCR [54] to handle content from scanned images. The extractions are passed through a post-processing model to detect the document language, correct the reading order, match figures with captions, and label metadata to output a structured Markdown file [47]. PyMuPDF4LLM [49] begins by opening the desired PDF document using PyMuPDF [55] and extracting text, text orientation, font properties, and designated clipping areas for multi-column layouts. This ensures the extraction process respects the natural reading order and layout specifics. Furthermore, PyMuPDF4LLM performs OCR to convert image-based text into editable text formats using Tesseract [56, 57]. The result of the parsing stage is a structured Markdown file that retains critical formatting and layout details. The output Markdown files generated serve as a standardized input for the LLMs for subsequent text analysis and data extraction.

In the image mode, the first step is to convert the PDF pages into images using the PyMuPDF [55] and the OpenCV [58] Python libraries. Once the PDF page images are obtained, their orientation is corrected to standardize the layout for text recognition. This involves converting the image to grayscale and detecting edges using the Canny algorithm [59], implemented through the OpenCV library. The resulting edges are then used to identify lines in the image using the Hough Line Transform. The angles of these lines are calculated to determine the dominant orientation of the text. Depending on the identified angle, the image is rotated to ensure that the text is aligned horizontally, enhancing the accuracy of subsequent text extraction. Following orientation correction, images are resized to ensure dimension uniformity, facilitating faster processing in later stages. The resizing uses the Pillow [60] Python library, which adjusts the image dimensions while maintaining the aspect ratio, using the Lanczos resampling method for high-quality output. The correctly oriented and resized images are converted to JPEG format before being encoded into base64 strings. This conversion is necessary to prepare the images for submission to a LLM provider API, which accepts base64-encoded images. The encoding process enables image data to be embedded directly within API requests, eliminating the need for external file storage [23, 61]. It is important to clarify that “image mode” refers to the process of converting entire PDF pages into images for analysis by a MLLM; this decision is not based on the number of figures or images the document contains.

To address the diverse challenges posed by different document types, we integrate two different approaches for PDF parsing: rule-based and learning-based. Rule-based tools, such as PyMuPDF4LLM, analyze PDF bytecode, including coordinates, fonts, and drawing commands, to extract structured content. In

contrast, learning-based tools employ machine learning models, including deep learning and MLLMs, to interpret document structure and content.

Learning-based approaches can be further subdivided into modular and end-to-end systems. In this workflow, both categories are implemented. Modular systems used in the workflow include GROBID, Marker, and Docling, which break down document parsing into specialized subtasks, such as layout analysis, OCR, and table or equation detection. End-to-end systems were employed in the image mode with MLLMs to process entire document pages or substantial regions within a unified neural network. Together, rule-based and learning-based approaches provide the flexibility necessary to navigate the complexity of heterogeneous scientific and technical documents. To support the evaluation of the PDF parsing step, our workflow enables users to review and download intermediate outputs, such as Markdown or JSON files, before LLM-based extraction.

After executing a data extraction analysis using the “New Analysis” mode, all selected options and prompts are stored as “flow variables” and exported as a configuration file (.variables), generated by the “Write Variables” node [62]. The configuration file is used as input for the “New Analysis with Prior Configuration” running mode. This mode uses all the configurations and prompts from a past analysis in a new set of files. The configuration file is read with the “Read Variables” node [63].

## 2.4 | Prompt Input and Large Language Models

Two prompt input options are available in the workflow to accommodate different user needs and levels of expertise: “Write custom prompt” and “Build schema visually”. In both modes, the interaction with LLMs is structured through interaction with the system prompt and the user prompt. The system prompt provides initial instructions to the model, setting the context and defining expected behavior (e.g., “You are an expert data extraction assistant”). The user prompt specifies the extraction instructions for each document, guiding the model in identifying and extracting the desired data from the PDFs. In the “Write custom prompt” mode, the workflow provides pre-populated text fields for both the system and user prompts, which users can edit to suit their specific needs. This mode offers maximum flexibility and control, allowing advanced users to compose free-form instructions tailored to complex or highly specialized extraction tasks. In our case studies, this mode was used. Alternatively, the “Build schema visually” mode enables users to define the variables to extract from PDFs through an intuitive graphical interface, eliminating the need to manually craft long or detailed prompts (Figure S1). Users can add, edit, remove, and nest variables (including strings, numbers, booleans, arrays, and objects) and provide human-readable descriptions for each field. The interface automatically generates a JSON schema based on the user’s selections, which is then passed to the LLM as structured guidance for extraction. In this mode, both the system and user prompts are optional; if left blank, defaults are applied to ensure consistent model behavior. The schema can also be imported or pasted if the user already has a predefined structure, further streamlining the setup for recurring tasks. The generated JSON schema, along with any optional prompts, is included

in the API call to the LLM, which is instructed to return only the extracted field names and their values in a clean JSON format.

In the text mode for scientific publications, the user prompts are combined with the JSON output generated by GROBID and one of the table extraction tools (tabula-py or Docling). In contrast, in the text mode for general PDF files, the Markdown file generated by one of the text extraction tools (Marker, Docling, or PyMuPDF4LLM) is combined with the user prompt before the input for the LLM. The base64-encoded images from the PDF pages are combined with the user prompt for the image mode.

The LLMs implemented in the workflow for text mode were Llama 3.1 (70B and 8B), Claude 3.5 Sonnet, Claude 3 Opus, Claude 3 Haiku, GPT-4o, and GPT-4o mini. In image mode, MLLMs, including Claude 3.5 Sonnet, Claude 3 Opus, Claude 3 Haiku, GPT-4o, and GPT-4o mini, were implemented. These models are accessed via APIs implemented through the LiteLLM [64] Python library. This framework facilitates integrating different models by calling all LLM APIs using the OpenAI format. Under the NIH network, the models’ APIs are accessed through the ToxPipe [65] framework, which hosts the free and local models (Llama 3.1 70B and 8B) and covers the costs for the API calls for the paid providers. For use outside the NIH network, the users must provide their API keys to the LLM providers. A detailed description of the models’ context window (maximum number of input tokens), the maximum number of output tokens, and current pricing is available in Table S1. For all models, the temperature parameter was set to 0. This parameter governs the randomness of the models’ predictions, with a value of 0 producing completely deterministic results and values closer to 1 or higher generating increasingly random and diverse outputs. The temperature setting is configured to 0 to minimize response variability and limit the models’ tendency toward creative outputs or hallucinations. This ensures more stable and predictable outputs, which is especially crucial for data extraction tasks that require high precision and consistency [66]. For all workflow execution options, the output of the extractions is in JSON format in a key-value organization, where the entities the user wants to extract are the keys and the extractions are the values. The JSON format was chosen because it is machine-readable and can be easily manipulated and converted to other formats [13]. The APIs of GPT-4o and GPT-4o mini models have the parameter “response\_format” that can be defined as “{‘type’: ‘json\_object’}” to force the output as a valid JSON. For the other models, the output in JSON is defined in the user prompt.

## 2.5 | Report and Download of Results

After one of the available LLMs performs the data extraction, the JSON output is directly available for download and, when possible, converted to a dataframe using the Pandas [67] Python library. Then, the Pandas dataframe is displayed on the workflow report screen of the user interface and converted to a downloadable CSV or XLSX file. The report and download of results page also provides the configuration file (.variables), helpful for those wishing to replicate the configurations and prompts of a past analysis, and a CSV file with details of the workflow execution including the PDF type (scientific publications or general PDFs), PDF extraction mode (text or image mode), the extraction

library for general PDFs (PyMuPDF4LLM, Marker, or Docling), and the LLM selected.

The workflow also allows highlighting the extracted variables in the original PDF file to facilitate human evaluation of the extractions. The texts of the extracted variables are compiled into a list of search targets, and the script iterates through each page of the PDF file, searching for occurrences of these target texts. When matches are found, the PyMuPDF [55] Python library creates yellow-highlight annotations over the identified text regions and attaches corresponding comments with the variable's name to provide contextual information. The modified PDF is saved as a new file that can be downloaded, producing a document enriched with clear and visually accessible highlights and comments corresponding to the extracted data.

## 2.6 | Documentation and User Guides

To help the user select the best options for their specific needs, we embedded guides directly within the KNIME workflow. Detailed comments were provided for every node, metanode, component, and Python script in the workflow, explaining their specific functions and purposes. Each step in the workflow execution included interactive user interface screens to improve user guidance and ease of use. Informative text boxes were incorporated at every stage to deliver clear and concise instructions, ensuring users were guided systematically through the different phases of the data extraction workflow.

## 2.7 | Case Studies

To showcase and test the options available in our workflow, we designed two case studies: (1) extraction of variables from peer-reviewed publications of toxicological studies and (2) extraction of variables from publicly available SDSs.

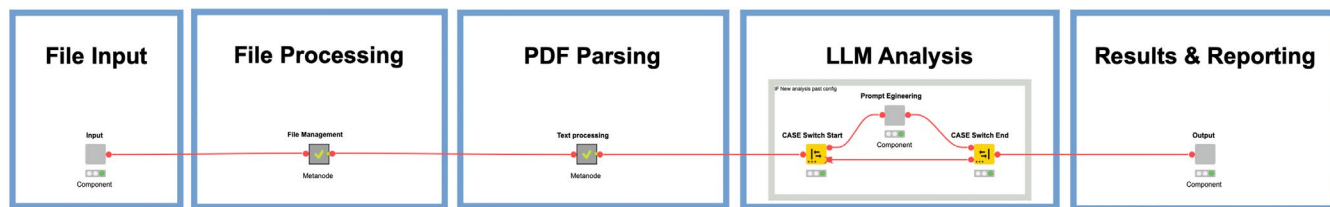
For the case study of peer-reviewed publications of toxicological studies, we evaluated the workflow using three randomly chosen publications [68–70]. The targeted variables for extraction included parameters such as animal information (e.g., species, age at treatment), administered doses, clinical observations during treatment (e.g., body weights, food consumption), and necropsy findings (e.g., organ weights, fetal examinations). The full list of extracted variables and their descriptions is available in the Table S2. For each variable, we prompted the LLMs in the workflow to determine its presence in the publication's text, extract the relevant entity, and record the source text from which the information was obtained. Using the text mode (GROBID for text extraction and tabula-py for table extraction), we tested the extractions using the models Claude 3 Opus, Claude 3.5 Sonnet, Claude 3 Haiku, GPT-4o, GPT-4o mini, Llama 3.1 70B, and Llama 3.1 8B. Using the image mode, we tested the extractions using the MLLMs Claude 3.5 Sonnet and GPT-4o. We engineered the prompts using 10 main core parts: (1) task definition: the opening statement explicitly defines the task; (2) output format: enforces strict JSON formatting; (3) JSON structure: detailed guidelines on how variables should be represented; (4) data handling: rules for handling missing or null values and

combining related variables into coherent strings; (5) value extraction guidelines: instructions for extracting specific types of variables; (6) specific variable handling: examples for complex variables (e.g., combining multiple related pieces of information such as species); (7) consistency: emphasizes uniform formatting; (8) error handling and reporting: instructions for ambiguous or missing data; (9) examples: examples illustrating expected outputs; and (10) variables: the list of variables to extract. The prompts used are available in the Table S3.

For the SDSs, we chose target variables related to mixture formulations (CAS numbers and active ingredient composition percentages) and six toxicity endpoints (e.g., skin sensitization, eye irritation, acute oral toxicity) as objects to extract. The full list of extracted variables from SDSs and their descriptions are available in the Table S4. We obtained the SDSs from the publicly accessible BASF Hub (<https://downloadcenter.basf.com/search>), accessed on 25 June 2024) using the filters: “Safety Data Sheet,” “English,” “Agriculture (129),” and “USA,” which resulted in 110 PDFs that were compressed into a ZIP file. The SDSs encompass 88 unique agrochemical substances, each SDS representing a mixture formulation that contains between one and nine active ingredients combined with excipients. Extractions were performed using the GPT-4o and Claude 3.5 Sonnet models for both the text and image modes. In this case study, PDF parsing for the text mode was done using Docling. The prompts used in both modes provided example outputs, available in Table S5. The prompts were partially engineered using OpenAI's API “beta” prompt generator [71], which can transform a complex task into simple steps for LLMs.

To evaluate the quality of the extractions and compare results across different models, we employed manual scoring. This approach was chosen because, to the best of our knowledge, no standardized dataset exists to benchmark NER approaches in the toxicology field. Additionally, NER in toxicology is inherently a fuzzy task, and relying solely on automated scoring methods, such as word matching, could underestimate model performance [20]. For example, in the extraction of the variable “route of administration”, if a model identifies “oral” while the publication specifies “gavage”, the extraction remains correct. However, an automated word-match validation would incorrectly flag it as an error. We adopted a scoring methodology based on work of Gartlehner et al. [72]. Specifically, we categorized the extractions as follows: true positives (TP), where entities were correctly extracted by the LLM from the original file; true negatives (TN), where the LLM accurately identified that the entity was not present in the original file; false positives (FP), where the LLM generated fabricated data that did not exist in the original file (i.e., hallucinated data); and false negatives (FN), where the LLM failed to extract or incorrectly extracted an entity from the original file. Subsequently, for each model's extractions, we calculated the total counts of TP, TN, FP, and FN and used these to derive the evaluation metrics: accuracy (Equation 1), precision (Equation 2), recall (Equation 3), and F1 (Equation 4). Accuracy measures the overall correctness of the extractions by the LLM. It calculates the proportion of correct extractions (both TP and TN) out of all extractions. Precision measures how well the LLM avoids generating false positives (hallucinated entities), i.e.,





**FIGURE 2** | Desktop version of the data extraction workflow.

the proportion of correctly extracted entities (TP) out of all entities the LLM identified as present (TP + FP). Recall measures how well the LLM identifies entities that are actually present in the original file, i.e., the proportion of correctly extracted entities (TP) out of all actual entities that should have been extracted (TP + FN). F1 is the harmonic mean of precision and recall, balancing the trade-off between the two metrics with a single score that reflects the LLM's performance in terms of both avoiding hallucinations (precision) and minimizing missed entities (recall).

$$\text{Accuracy} = \left( \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}} \right) \quad (1)$$

$$\text{Precision} = \left( \frac{\text{TP}}{\text{TP} + \text{FP}} \right) \quad (2)$$

$$\text{Recall} = \left( \frac{\text{TP}}{\text{TP} + \text{FN}} \right) \quad (3)$$

$$\text{F1} = \left( 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \right) \quad (4)$$

### 3 | Results and Discussion

#### 3.1 | Workflow Overview

The data extraction workflow is executed in 5 main steps: (1) ZIP file input containing the PDF files; (2) selection of the PDF type (scientific publication or general PDFs); (3) selection of extraction mode (text or image); (4) prompt input; and (5) results visualization and download. In the desktop version, the graphical user interfaces for each step are accessed through interaction with the KNIME components developed (Figure 2). The graphical user interfaces for steps 1 through 3 are shown in Figure 3. Prompt input and results visualization are shown in Figures 4 and 5, respectively.

The workflow is designed for use through its GUI, guided by in-workflow documentation. This documentation aims to provide clear and concise instructions, including the logic behind each step, required user input, and any potential parameter adjustments. Each user interface screen of the workflow includes a text box on the right side, providing instructions for executing the workflow and guidance for interpreting the output. This structured guidance ensures that users can easily navigate the workflow and understand the methodology being applied. Thus, the documentation strategy not only ensures ease of use but also supports reproducibility and potential future modifications or extensions to the workflow.

#### 3.2 | Data Extraction From Scientific Publications

##### 3.2.1 | Workflow Execution

To test the workflow on extracting data from scientific publications, we used three published toxicological studies [68–70]. The three English-language publications described reproductive or developmental toxicity studies in small animal models (i.e., mice, rats, or rabbits) and each was published in a different journal and on a different chemical. This relatively focused dataset was appropriate for the goals of this study. However, we recognize that the studies represent a small subset of the diversity of study designs, endpoints, chemicals, and analytical approaches that are relevant to the field of toxicology.

In our case studies, the “New Analysis” mode was selected (Figure 3). On the initial screen of the data extraction workflow, we then uploaded the ZIP file containing the PDF publications. Then, in the “PDF type” field, we selected the “Scientific publications” option. When this option is selected, the “Table extraction method” is displayed in the user interface screen. For our case study, we used Tabula-py to extract the tabular data from the publications. After this, the next field available is the “PDF extraction mode” with the “text mode” and “image mode” options. Here, we made different executions of the workflow to test both text and image modes. When selecting the “text mode”, the LLM options available are “GPT-4o”, “GPT-4o mini”, “Claude 3.5 Sonnet”, “Claude 3 Opus”, “Claude 3 Haiku”, “Llama 3.1 70B”, and “Llama 3.1 8B”. All these models were used in our case study. When selecting the “image mode”, the MLLM options available are “Claude 3.5 Sonnet”, “Claude 3 Opus”, “Claude 3 Haiku”, “GPT-4o”, and “GPT-4o mini” (Figure S2). In our case study, the MLLMs tested were “Claude 3.5 Sonnet” and “GPT-4o”.

After selecting the “PDF type”, “PDF extraction mode”, “Table extraction method” (only for the text mode), and the LLM model, the following user interface screen is the prompt input (Figure 4). Prompt engineering is one of the most important components in the application of LLMs for automated data extraction [66]. The design and structure of a prompt significantly influence the accuracy, consistency, and reliability of the model's outputs. A well-crafted prompt ensures that the LLM focuses on the specified task while reducing ambiguity, thus enhancing the precision and relevance of the extracted data. Furthermore, including explicit instructions for handling missing or unclear data in the prompt mitigates uncertainties and contributes to the robustness of the extraction process [73, 74].





# Data Extraction Workflow

## Analysis Type Selection

☒ Start New Analysis

☐ Start New Analysis with Prior Configuration

## Zip File Upload

Use the file browser below to upload the .zip file containing PDFs.

Choose File

Scientific\_Publications.zip

Size: 1.74 MB

## PDF type

☒ Scientific publications

☐ General PDFs

## PDF extraction mode

☒ Text Mode (Extract text from PDF)

☐ Image mode (Analyze PDF pages as images)

## Table extraction method

☒ tabula-py

☐ Docling

## LLM Model

☒ GPT-4o

☐ GPT-4o mini

☐ Claude 3.5 Sonnet

☐ Claude 3 Opus

☐ Claude 3 Haiku

☐ Llama 3.1 70B

☐ Llama 3.1 8B

## Guide

### 1. Analysis Type

**Start New Analysis:** If you're looking to perform a new analysis, click on the "Start New Analysis" button.

**Start New Analysis with Prior Configuration:** If you'd like to apply the settings from a previous run to new files, select this option. This option requires a saved configuration file.

### 2. File Upload

Upload a ZIP file containing the PDF files you want to process.

### 3. PDF type

The workflow offers two modes for extracting data from scientific publications:

- Scientific Publications:** The workflow applies methods for documents that follow the IMRAD structure (Introduction, Methods, Results, and Discussion).
- General PDFs:** The workflow applies methods for documents which may not necessarily follow the IMRAD format or require OCR (Optical Character Recognition) processing.

### 4. Table extraction method

The workflow offers two modes for extracting tables from scientific publications:

- tabula-py:** tabula-py is a Python wrapper for the Java-based Tabula tool (which uses PDFBox) to extract tables from PDFs.
- Docling:** Uses the advanced TableFormer AI model to extract tables from PDFs.

### 5. PDF extraction mode

The workflow offers two modes for extracting data from scientific publications:

- Text Mode:** Extracts the text from the PDF file and processes it using a Large Language Model (LLM) to extract the required data. \*Does not perform OCR.
- Image Mode:** Converts PDF pages to images and uses a Large Language Model with vision capabilities to extract the required data.

### 6. Large Language Models

Choose one of the LLM options available: "GPT-4o", "GPT-4o mini", "Claude 3.5 Sonnet", "Claude 3 Opus", "Claude 3 Haiku", "Llama 3.1 70B", and "Llama 3.1 8B".

Details

**FIGURE 3** | Screen of the user interface for ZIP file input and selection of PDF type, PDF extraction mode, table extraction method, and selection of the LLM.

9 of 16

17590884, 2025, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/wcms.70047 by Capes, Wiley Online Library on [26/04/2025]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



# Data Extraction Workflow

## Prompt Input Method

- ☒ Write custom prompt  
☐ Build schema visually

## Prompts

### System prompt

You are a scientific assistant and toxicologist expert in reviewing scientific publications and extract and analyze specific relevant information out of PDFs in valid JSON format. Be careful with numbers.

### User prompt - instruction

Analyze the input PDFs and extract the specified variables to create a new JSON output. Follow these guidelines:

1. Output Format:
  - Return only a valid JSON object.
  - Do not include any additional text before or after the JSON output.
2. JSON Structure:
  - Each variable becomes a key-value pair in the JSON object.
  - Use the exact variable names provided in the list below.
3. Data Handling:
  - Use 'N/A' for missing or null values.
  - Ensure uniform formatting for similar types of variables.
  - Include only the specified variables in the output.
  - Combine related variables into a single value without nesting (e.g., combine "specie 1" and "specie 2" into a single "Animal" value).
4. Value Extraction Guidelines:
  - For 'Y/N' variables: Use 'Y' if information is present, 'N' if absent.
  - For entity variables: Provide specific values or ranges (e.g., '8 weeks', '200 grams').
  - For source text variables: Provide the exact relevant text snippet from the input.
5. Specific Variable Handling:
  - 'Animal': Provide species as a single string (e.g., 'Rat', 'Mouse'). Separate multiple species with commas.

[JSON\\_files](#) (zip)

## Guide

### Prompt Input Methods

Choose between two methods for defining your LLM prompts:

- **Write custom prompt:** Manually write your own system and user prompts with complete control over the instructions.
- **Build schema visually:** Use the interactive schema builder to create structured JSON schemas that automatically generate optimized prompts for data extraction.

### Prompts

The interaction with LLMs is performed through prompts defined by two components: system and user prompts.

- **System prompt:** Initial instructions for models as context and clues on expected behavior.
- **User prompt:** Is the user's input, specifying the instructions they want the model to address for each document and guiding them in extracting the desired data from the PDFs.

### JSON files

GROBID extracts the main text of the scientific publication and save them as JSON files. These files can be downloaded using the "JSON\_files" button.

**FIGURE 4** | Screen of the user interface for prompt input using the “Write custom prompt” option.

The prompts used in this case study were carefully designed using the “Write custom prompt” option to guide the LLMs in extracting structured, accurate, and domain-specific data. We used the same prompt for all the LLMs. The text and image modes had only slight differences in the beginning of their user prompts to specify the input of JSON or images. The prompts used in this case study are available in the Table S3. The system prompt establishes the model's role as a scientific assistant with expertise in toxicology, emphasizing precision, particularly

in numerical data handling. This foundational instruction ensures that the model operates within a contextual framework aligned with the needs of toxicological data extraction. The user prompt provides detailed guidelines for output formatting, data structuring, and specific variable handling. It mandates the generation of a valid JSON object, ensuring that the output is directly machine-readable. Comprehensive instructions for data handling, including the use of placeholders such as “N/A” for missing values and the requirement for consistent formatting,



## Data Extraction Workflow

### Data Extraction Results

[Download \(csv\)](#)

[Download \(xlsx\)](#)

[JSON\\_output\\_files \(zip\)](#)

Show  entries

Search:

| Title                                                                                                       | Animal               | Age at treatment (Y/N) | Age at treatment (Entity) | Age at treatment (Source text)                                         | Body weight at treatment? (Y/N) | Body weight at treatment (Entity)    | Body weight at treatment (Source text)                                                        | Number of animals per treatment group (note if this is given as #M and #F, or # overall evenly split between the two) | Route of administration |
|-------------------------------------------------------------------------------------------------------------|----------------------|------------------------|---------------------------|------------------------------------------------------------------------|---------------------------------|--------------------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|-------------------------|
| Developmental toxicity of polyethylene glycol-g-polyvinyl alcohol grafted copolymer in rats and rabbits     | Wistar rats, Rabbits | Y                      | 96-103 days old           | On GD 0, females were about 96-103 days old                            | Y                               | 207.9 ± 13.4 g                       | On GD 0, females were about 96-103 days old with a mean body weight of 207.9 ± 13.4 g         | 25 females per dose group                                                                                             | Gavage                  |
| Effect of Astragaloside IV on the General and Peripartum Reproductive Toxicity in Sprague-Dawley Rats       | Sprague-Dawley rats  | N                      | N/A                       | N/A                                                                    | Y                               | Females: 180-230 g, Males: 200-250 g | nulliparous females weighing 205 + 25 g (180-230 g) and males weighing 225 + 25 g (200-250 g) | ?                                                                                                                     | Intravenous             |
| Effect of grape seed extract on maternal toxicity and in utero development in mice treated with zearalenone | Swiss albino mice    | Y                      | 10-12 weeks               | Adult female and male Swiss albino mice (10-12 weeks old, 25-30 g b.w) | Y                               | 25-30 g                              | Adult female and male Swiss albino mice (10-12 weeks old, 25-30 g b.w)                        | ?                                                                                                                     | Oral                    |

Showing 1 to 3 of 3 entries

Previous **1** Next

### Selected Options

[Download \(csv\)](#)

### Configuration File

[Download \(variables\)](#)

### Guide

#### Results and Report

After the data extraction performed by one of the available LLMs performs the data extraction, the JSON output can be downloaded and converted to a data frame using the Pandas Python library. Then, the Pandas data frame is displayed in on the workflow report page and converted to a CSV and/or XLSX files that can be also downloaded. In addition to the extractions' results, this page also provides the Configuration file (.variables), helpful for those wishing to replicate the configurations and prompts of a past analysis, and a CSV file with details of the workflow execution including the PDF type (scientific publications or general PDFs), PDF extraction mode (text or image mode), the extraction library for general PDFs (PyMuPDF4LLM, Marker, or Docling), and the LLM selected.

FIGURE 5 | Report and results download screen of the user interface.

further enhanced the reliability of the extracted data. The inclusion of specific rules for interpreting and consolidating complex variables, such as combining related species into a single value or formatting doses with units, was important for standardizing the extractions of some variables. The inclusion of output examples anchored the model's interpretation of the expected output and enhanced the ability to handle edge cases effectively. This approach also incorporated error-handling mechanisms, specifying clear actions for incomplete or ambiguous data, which strengthens the robustness of the workflow. At the end of the prompt, we provided the list of 76 variables we wanted the models to extract from the publications (Table S3).

After the prompt engineering and input steps, the next user interface screen is used for visualizing and downloading the results (Figure 5). On this screen, a table displays the extracted variables for all the uploaded publications, allowing for navigation and review. This table can be downloaded in CSV or XLSX formats using the respective "Download" button. In addition, the JSON output files from the LLM extractions can be downloaded using the "JSON\_output\_files" button. In this case study, for each tested LLM in the text and image modes, we downloaded the respective CSV files. In the same page, the user can also download the configuration file (.variables) and a CSV file with details of the workflow execution (all the selected options and prompts).

### 3.2.2 | Model Testing and Performance

The downloaded CSV files for each tested LLM model and execution mode were used for manual scoring, comparing the model's extractions with those of humans. Due to the number of variables being extracted (76 per tested publication resulting in 228 variables per model), the number of tested models, and the use of manual scoring, this initial proof-of-concept study was limited to three publications. The extractions and the manual scorings are available in the Supporting Information File S1.

After manual scoring and the calculation of validation metrics, among the models evaluated in text mode, Claude 3 Opus and Claude 3.5 Sonnet achieved the best overall performance, with

identical results: an accuracy of 81.14%, precision of 100%, recall of 73.46%, and an F1 of 84.70% (Table 1). The 100% precision scores observed for Claude 3 Opus and Claude 3.5 Sonnet are a direct result of the prompt engineering and model configuration strategy employed in this study. To ensure deterministic outputs and prevent hallucination, the temperature parameter was set to zero, and the user prompts explicitly instructed the models to return "N/A" when information was absent or could not be confidently extracted. This approach effectively eliminated False Positives, defined as fabricated or hallucinated data not present in the source text. With an FP count of zero, the precision formula (Equation 2) yields a 100% score, reflecting the success of this strategy in avoiding fabricated outputs. However, the recall scores capture the model's tendency to miss available information (False Negatives) rather than fabricate it. For example, in the extractions made by Claude 3 Opus, the assigned scores were TP: 119, TN: 66, FP: 0, and FN: 43. Using the recall formula (Equation 3), the resulting score of 73.5% highlights this tendency.

GPT-4o demonstrated comparable performance, with the same accuracy of 81.14%, a slightly lower precision of 99.16%, but a marginally higher recall of 73.75%, resulting in an F1 of 84.59%. Notably, GPT-4o offers lower costs compared to Claude 3 Opus and Claude 3.5 Sonnet, making it a potentially more cost-effective choice depending on the user's objectives. GPT-4o mini also performed well in text mode, with an accuracy of 79.39% and an F1 of 83.62%, albeit with lower precision (95.24%) and slightly higher recall (74.53%). In image mode, the models generally underperformed compared to text mode, reflecting the additional challenges posed by extracting information from complex document layouts. Claude 3.5 Sonnet achieved the highest performance in this mode, with an accuracy of 67.56%, precision of 71.05%, recall of 78.83%, and an F1 score of 74.74%. GPT-4o demonstrated similar results, with an accuracy of 64.44%, precision of 72.48%, recall of 73.47%, and an F1 of 72.97%. The text mode, which filters the extracted text to include only the main sections of IMRAD-formatted publications, while removing references, author lists, funding information, and other secondary content, demonstrated consistently higher performance compared to image mode, mainly because it presents to the models only the relevant sections for the extractions. Despite these

**TABLE 1** | Overall performance of LLMs and MLLMs on data extraction from scientific publications of toxicological studies using the text and image modes.

| Extraction mode | Model             | Accuracy | Precision | Recall | F1    |
|-----------------|-------------------|----------|-----------|--------|-------|
| Text            | Claude 3 Opus     | 81.14    | 100       | 73.46  | 84.7  |
|                 | Claude 3.5 Sonnet | 81.14    | 100       | 73.46  | 84.7  |
|                 | GPT-4o            | 81.14    | 99.16     | 73.75  | 84.59 |
|                 | GPT-4o mini       | 79.39    | 95.24     | 74.53  | 83.62 |
|                 | Llama 3.1 70B     | 71.49    | 90.65     | 63.82  | 74.9  |
|                 | Llama 3.1 8B      | 70.61    | 86.61     | 68.75  | 76.65 |
|                 | Claude 3 Haiku    | 57.46    | 89.84     | 57.79  | 70.34 |
| Image           | Claude 3.5 Sonnet | 67.56    | 71.05     | 78.83  | 74.74 |
|                 | GPT-4o            | 64.44    | 72.48     | 73.47  | 72.97 |



results, image mode provides a viable alternative for extracting information from documents with significant graphical content or non-textual layouts. These findings highlight the trade-offs between cost and performance in selecting the optimal model and mode for a given application.

### 3.3 | Data Extraction From General PDFs—Safety Data Sheets

#### 3.3.1 | Workflow Execution

Although peer-reviewed literature is a widely used source of reliable data in toxicology, a plethora of data is also published by companies and regulatory agencies in the form of technical reports or safety data sheets, serving as support documents for regulatory decisions, such as classification and labeling requirements. Thus, the second case study of this workflow was performed using general PDFs, which do not necessarily follow the IMRAD format. For these files, since their layout structure can be in different formats or require OCR, the text mode extracts the full text content for subsequent data extraction. The image mode follows the same methodology in both “Scientific publications” and “General PDFs” options. Here, we utilized 110 SDSs from the publicly accessible BASF Hub to evaluate data extraction from general PDFs. SDSs are documents that provide comprehensive information about chemical products and include information such as the properties of each chemical; the physical, human health, and environmental hazards; protective measures; and safety precautions for handling, storing, and transporting the chemical. Our goal in using these SDSs was to extract toxicological data for chemical mixtures, which have scarce machine-readable datasets in the literature, to support the future creation of a database ready to be used in machine learning projects.

To make the extractions for general PDFs, after the input of the ZIP file containing the PDFs of the SDSs in the first user interface screen of the data extraction workflow, the user must select in the “PDF type” field, the “General PDFs” option. Similar to the case study for scientific publications, we tested both text and image modes. When the “text mode” is selected for general PDFs, the “Extraction library” field is shown with the options “PyMuPDF4LLM”, “Marker”, and “Docling”. We used the Docling option for our text mode test. The LLM options available are “GPT-4o”, “GPT-4o mini”, “Claude 3.5 Sonnet”, “Claude 3 Opus”, “Claude 3 Haiku”, “Llama 3.1 70B”, and “Llama 3.1 8B”. In this case, to save costs and based on prior performance metrics, we tested only the models Claude 3.5 Sonnet and GPT-4o for both text and image modes. The user interface screens for

prompt input, report, and results download are the same for both “Scientific publications” and “General PDFs” options.

The prompts for the text and image modes in this study were designed to systematically guide the extraction of structured data from the SDSs in a JSON output format. These prompts are available in the Table S5. In both modes, the prompts emphasize the identification and parsing of relevant sections such as formulation identification, composition, and toxicological information. The text mode prompt is structured to process Markdown files and provides detailed extraction instructions with specific examples for the output format but not for the input. In preliminary tests, the use of examples for the input in the text mode resulted in hallucinations in the models' extractions, where the models tended to use the examples in the output (data not shown). In contrast, the image mode prompt had a structure to handle images by incorporating additional steps for identifying and parsing visual content and included examples of both input and output formats. Both prompts maintain consistency in variable naming and require placeholders for missing data. Here, in total, we prompted the model to extract 46 variables from each of the SDSs (Table S4).

#### 3.3.2 | Model Testing and Performance

After the manual scoring of all the models' extractions, the performance from general PDF files demonstrates a notable performance difference between the text and image modes. The text mode exhibited poor performance overall, potentially due to limitations in the layout identification process and the conversion of extracted content into Markdown files. Claude 3.5 Sonnet achieved an accuracy of 49.97%, precision of 99.75%, recall of 14.76%, and an F1 of 25.71%, while GPT-4o achieved slightly higher metrics with an accuracy of 50.59%, precision of 94.63%, recall of 17.16%, and an F1 of 29.05% (Table 2). These results indicate that while precision was high, both models had significant difficulty identifying all relevant entities, as reflected in the low recall values. In contrast, the image mode outperformed the text mode substantially, leveraging multimodal capabilities to process the complex layouts of SDSs. Claude 3.5 Sonnet achieved an accuracy of 96.73%, precision of 99.74%, recall of 96.98%, and an F1 of 98.34%, while GPT-4o exhibited even better performance, with an accuracy of 98.54%, precision of 99.39%, recall of 99.14%, and an F1 of 99.14% (Table 2). The extractions and manual scorings are available in Supporting Information File S2.

These findings underscore the importance of selecting the appropriate extraction mode based on the document type. For

**TABLE 2** | Overall performance of LLMs and MLLMs on data extraction from safety data sheets using the text and image modes.

| Extraction mode | Model             | Accuracy | Precision | Recall | F1    |
|-----------------|-------------------|----------|-----------|--------|-------|
| Text            | Claude 3.5 Sonnet | 49.97    | 99.75     | 14.76  | 25.71 |
|                 | GPT-4o            | 50.59    | 94.63     | 17.16  | 29.05 |
| Image           | Claude 3.5 Sonnet | 96.73    | 99.74     | 96.98  | 98.34 |
|                 | GPT-4o            | 98.54    | 99.39     | 99.14  | 99.14 |

general PDFs with diverse and complex layouts, such as SDSs, the image mode proved far superior, with near-perfect performance across all evaluation metrics. In contrast, the text mode was limited by the inherent challenges of unstructured text and the need for documents with clear and linear text layouts. This mode may not capture spatial relationships or visual cues effectively, which are often crucial for understanding the full context of the document, leading to high precision but very low recall and overall performance. This analysis highlights the value of multimodal approaches in addressing the challenges posed by non-standardized document formats.

## 4 | Conclusions

The data extraction KNIME workflow presented in this study demonstrates a robust and flexible solution for automating the extraction of key entities from scientific literature and general PDF files, here demonstrated with respect to toxicology but generalizable across fields. By integrating advanced large language models with tailored prompt engineering, the workflow addresses the inherent challenges posed by diverse document formats and complex layouts, achieving high performance across accuracy, precision, recall, and F1 metrics. The workflow performs two distinct parsing modes: text mode and image mode, each tailored to optimize data retrieval based on document structure. The text mode demonstrated strong performance when processing standard scientific publications following the IMRAD format, leveraging structured text parsing and LLM-based named entity recognition to achieve high accuracy and precision. In contrast, the image mode, leveraging multimodal capabilities, exhibited near-perfect performance in extracting data from visually complex regulatory support documents such as safety data sheets, highlighting the importance of selecting appropriate modes based on document structure. The use of a dual-mode approach, text or image-based extractions, provides a comprehensive and adaptable framework capable of handling diverse document types. The use of LLMs, including GPT-4o and Claude 3.5 Sonnet, further enhances the workflow's efficiency and scalability, offering cost-effective solutions without compromising performance. By integrating these state-of-the-art models into a user-friendly, low-code environment like KNIME, the workflow lowers barriers to adoption, empowering users with minimal technical expertise to perform automated data extraction tasks. Beyond toxicology, the customizable prompts and versatile design of this workflow position it as a valuable tool for broader applications across scientific domains. By facilitating the creation of high-quality, machine-readable datasets, this approach has the potential to accelerate advancements in research, enhance information retrieval and synthesis, and support the development and validation of new computational methodologies. Future work could focus on expanding the scope of extracted variables, refining prompts for even greater accuracy, testing the approach on a broader set of toxicology studies (i.e., covering greater diversity in animal or in vitro models, experimental design, exposures, etc.), and incorporating additional LLMs or multimodal capabilities as they evolve. The continued development and open accessibility of this workflow represent a significant step toward democratizing automated data extraction, ultimately fostering innovation and collaboration in toxicology and beyond.

## Author Contributions

**José Teófilo Moreira-Filho:** conceptualization (equal), data curation (equal), investigation (equal), methodology (equal), software (equal), validation (equal), visualization (equal), writing – original draft (equal), writing – review and editing (equal). **Dhruv Ranganath:** data curation (equal), methodology (equal), software (equal), validation (equal), writing – original draft (equal), writing – review and editing (equal). **Ricardo S. Tieghi:** data curation (equal), methodology (equal), software (equal), validation (equal), writing – original draft (equal), writing – review and editing (equal). **Robert Patton:** conceptualization (equal), methodology (equal), software (equal), writing – review and editing (equal). **Vicki Sutherland:** conceptualization (equal), methodology (equal), writing – review and editing (equal). **Charles Schmitt:** conceptualization (equal), methodology (equal), writing – review and editing (equal). **Andrew A. Rooney:** conceptualization (equal), methodology (equal), writing – review and editing (equal). **Jennifer Fostel:** conceptualization (equal), methodology (equal), writing – review and editing (equal). **Vickie R. Walker:** conceptualization (equal), methodology (equal), writing – review and editing (equal). **Trey Saddler:** methodology (equal), software (equal), writing – review and editing (equal). **David Reif:** conceptualization (equal), methodology (equal), resources (equal), writing – review and editing (equal). **Kamel Mansouri:** conceptualization (equal), funding acquisition (equal), methodology (equal), project administration (equal), resources (equal), software (equal), supervision (equal), writing – original draft (equal), writing – review and editing (equal). **Nicole Kleinstreuer:** conceptualization (equal), funding acquisition (equal), methodology (equal), project administration (equal), resources (equal), supervision (equal), writing – original draft (equal), writing – review and editing (equal).

## Acknowledgments

Portions of the proofreading and language editing were assisted by a large language model (ChatGPT 4). The authors reviewed and are responsible for all content and conclusions.

## Conflicts of Interest

The authors declare no conflicts of interest.

## Data Availability Statement

The data used in the case study and the chemical grouping workflow are available for download in the KNIME Community Hub at <https://hub.knime.com/s/oh8vLkMh-2h0th6B> and GitHub at [https://github.com/NIEHS/Data\\_extraction\\_workflow](https://github.com/NIEHS/Data_extraction_workflow). GitHub also provides a tutorial for installation.

## Related WIREs Articles

Generative models for molecular discovery: Recent advances and challenges.

## References

1. M. Ware and M. Mabe, *The STM Report: An Overview of Scientific and Scholarly Journal Publishing* (International Association of Scientific, Technical and Medical Publishers, 2015).
2. S. Fortunato, C. T. Bergstrom, K. Börner, et al., “Science of Science,” *Science* 359 (2018): 359, <https://doi.org/10.1126/science.aao0185>.
3. Z. Hong, L. Ward, K. Chard, B. Blaiszik, and I. Foster, “Challenges and Advances in Information Extraction From Scientific Literature: A Review,” *JOM* 73 (2021): 3383–3400.
4. Z. Hong, R. Tchoua, K. Chard, and I. Foster, “SciNER: Extracting Named Entities From Scientific Literature,” in *International Conference*

- on *Computational Science*, vol. 12138 (Springer Nature Switzerland AG, 2020), 308–321.
5. E. A. Olivetti, J. M. Cole, E. Kim, et al., “Data-Driven Materials Research Enabled by Natural Language Processing and Information Extraction,” *Applied Physics Reviews* 7 (2020): 041317.
6. A. B. Daniel, N. Choksi, J. Abedini, et al., “Data Curation to Support Toxicity Assessments Using the Integrated Chemical Environment,” *Frontiers in Toxicology* 4 (2022): 987848.
7. D. E. Hines, S. Bell, X. Chang, K. Mansouri, D. Allen, and N. Klein-streuer, “Application of an Accessible Interface for Pharmacokinetic Modeling and in Vitro to in Vivo Extrapolation,” *Frontiers in Pharmacology* 13 (2022): 864742.
8. J. Abedini, B. Cook, S. Bell, et al., “Application of New Approach Methodologies: ICE Tools to Support Chemical Evaluations,” *Computational Toxicology* 20 (2021): 100184.
9. C. Foster, J. Wignall, S. Kovach, et al., “Standardizing Extracted Data Using Automated Application of Controlled Vocabularies,” *Environmental Health Perspectives* 132 (2024): 027006.
10. S. M. Bell, J. Phillips, A. Sedykh, et al., “An Integrated Chemical Environment to Support 21st-Century Toxicology,” *Environmental Health Perspectives* 125 (2017): 054501.
11. M. P. F. Corradi, A. M. de Haan, B. Staumont, et al., “Natural Language Processing in Toxicology: Delineating Adverse Outcome Pathways and Guiding the Application of New Approach Methodologies,” *Biomaterials and Biosystems* 7 (2022): 100061.
12. A. Gupta, D. Mittal, O. Goel, and S. K. Jha, “Natural Language Processing Algorithms for Domain-Specific Data Extraction in Material Science: Reseractor,” *Journal of Materials Science* 59 (2024): 13856–13872.
13. L. Foppiano, G. Lambard, T. Amagasa, and M. Ishii, “Mining Experimental Data From Materials Science Literature With Large Language Models: An Evaluation Study,” *Science and Technology of Advanced Materials: Methods* 4 (2024): 2356506.
14. A. Birhane, A. Kasirzadeh, D. Leslie, and S. Wachter, “Science in the Age of Large Language Models,” *Nature Reviews Physics* 5 (2023): 277–280.
15. K. M. Jablonka, Q. Ai, A. Al-Feghali, et al., “14 Examples of How LLMs Can Transform Materials Science and Chemistry: A Reflection on a Large Language Model Hackathon,” *Digital Discovery* 2 (2023): 1233–1250.
16. L. Ying, Z. Liu, H. Fang, et al., “Text Summarization With ChatGPT for Drug Labeling Documents,” *Drug Discovery Today* 29 (2024): 104018.
17. A. D. White, “The Future of Chemistry Is Language,” *Nature Reviews Chemistry* 7 (2023): 1–2.
18. J. Clusmann, F. R. Kolbinger, H. S. Muti, et al., “The Future Landscape of Large Language Models in Medicine,” *Communication & Medicine* 3 (2023): 141.
19. W. Zhang, Q. Wang, X. Kong, et al., “Fine-Tuning Large Language Models for Chemical Text Mining,” *Chemical Science* 15 (2024): 10600–10611.
20. J. Dagdelen, A. Dunn, S. Lee, et al., “Structured Information Extraction From Scientific Text With Large Language Models,” *Nature Communications* 15 (2024): 1418.
21. M. P. Polak, S. Modi, A. Latosinska, et al., “Flexible, Model-Agnostic Method for Materials Data Extraction From Text Using General Purpose Language Models,” *Digital Discovery* 3 (2024): 1221–1235.
22. W. J. Gordon and R. S. Rudin, “Why APIs? Anticipated Value, Barriers, and Opportunities for Standards-Based Application Programming Interfaces in Healthcare: Perspectives of US Thought Leaders,” *JAMIA Open* 5 (2022): oao023.
23. M. Schilling-Wilhelmi, M. Ríos-García, S. Shabih, et al., “From Text to Insight: Large Language Models for Chemical Data Extraction,” *Chemical Society Reviews* 54 (2024): 1125–1150.
24. M. R. Berthold, N. Cebon, F. Dill, et al., “KNIME—The Konstanz Information Miner: Version 2.0 and Beyond,” *ACM Sigkdd Explorations Newsletter* 11 (2009): 26–31.
25. M. R. Berthold, N. Cebon, F. Dill, et al., “KNIME: The Konstanz Information Miner,” in *Data Analysis, Machine Learning and Applications*, ed. H. B. Preisach, L. Schmidt-Thieme, and R. Decker (Springer, 2008), 319–326.
26. J. T. Moreira-Filho, D. Ranganath, M. Conway, C. Schmitt, N. Klein-streuer, and K. Mansouri, “Democratizing Cheminformatics: Interpretable Chemical Grouping Using an Automated KNIME Workflow,” *Journal of Cheminformatics* 16 (2024): 101.
27. N. Livathinos, C. Berrospi, M. Lysak, et al., “Robust PDF Document Conversion Using Recurrent Neural Networks,” *Proceedings of the AAAI Conference on Artificial Intelligence* 35 (2021): 15137–15145.
28. H. Déjean and J.-L. Meunier, “A System for Converting PDF Documents Into Structured XML Format,” *Lecture Notes in Computer Science* 3872 (2006): 129–140.
29. H. Bast and C. Korzen, “A Benchmark and Evaluation for Text Extraction From PDF,” in *2017 ACM/IEEE Joint Conference on Digital Libraries (JCDL)* (IEEE, 2017), 1–10.
30. L. Markewich, H. Zhang, Y. Xing, et al., “Segmentation for Document Layout Analysis: Not Dead Yet,” *International Journal on Document Analysis and Recognition (IJDAR)* 25 (2022): 67–77.
31. D. Nadeau and S. Sekine, “A Survey of Named Entity Recognition and Classification,” *Linguisticae Investigationes* 30 (2007): 3–26.
32. A. S. Alshehri, K. A. Horstmann, and F. You, “Versatile Deep Learning Pipeline for Transferable Chemical Data Extraction,” *Journal of Chemical Information and Modeling* 64 (2024): 5888–5899.
33. J. Choi and B. Lee, “Accelerating Materials Language Processing With Large Language Models,” *Communications Materials* 5 (2024): 13.
34. A. Zhang, H. Rangwala, B. Pfizmann, et al., “DocLayNet: A Large Human-Annotated Dataset for Document-Layout Segmentation,” in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Association for Computing Machinery, 2022), 3743–3751.
35. M. Zhu and J. M. Cole, “PDFDataExtractor: A Tool for Reading Scientific Text and Interpreting Metadata From the Typeset Literature in the Portable Document Format,” *Journal of Chemical Information and Modeling* 62 (2022): 1633–1643.
36. C. Auer, M. Dolfi, A. Carvalho, C. B. Ramis, and P. W. J. Staar, “Delivering Document Conversion as a Cloud Service With High Throughput and Responsiveness,” in *2022 IEEE 15th International Conference on Cloud Computing (CLOUD)* (IEEE, 2022), 363–373.
37. D. Huang, C. Yan, Q. Li, and X. Peng, “From Large Language Models to Large Multimodal Models: A Literature Review,” *Applied Sciences* 14 (2024): 5068.
38. S. Yin, C. Fu, S. Zhao, et al., “A Survey on Multimodal Large Language Models,” *National Science Review* 11 (2024): nwae403.
39. “GROBID,” accessed December 5, 2024, <https://github.com/kermitt2/grobid>.
40. “ElementTree,” accessed December 27, 2024, <https://github.com/python/cpython/blob/3.13/Lib/xml/etree/ElementTree.py>.
41. L. Angélica, S. Lattanzi, A. M. Medina, et al., “Table Meets LLM: Can Large Language Models Understand Structured Table Data? A Benchmark and Empirical Study,” in *Proceedings of the 17th ACM International Conference on Web Search and Data Mining* (Association for Computing Machinery, 2024), 645–654.



42. D. Circi, G. Khalighinejad, A. Chen, B. Dhingra, and L. C. Brinson, "How Well Do Large Language Models Understand Tables in Materials Science?," *Integrating Materials and Manufacturing Innovation* 13 (2024): 669–687.
43. "pdfplumber," accessed December 5, 2024, <https://github.com/jsvine/pdfplumber>.
44. "tabula-py," accessed December 5, 2024, <https://github.com/chezou/tabula-py/tree/master>.
45. "nougat," accessed May 12, 2025, <https://github.com/facebookresearch/nougat>.
46. "marker," accessed December 5, 2024, <https://github.com/VikParuchuri/marker>.
47. "docling," accessed May 12, 2025, <https://github.com/docling-project/docling>.
48. "Camelot," accessed December 5, 2024, <https://github.com/atlanhq/camelot>.
49. "PyMuPDF4LLM," accessed December 12, 2024, <https://github.com/pymupdf/RAG/tree/main/pymupdf4llm>.
50. "Surya," accessed December 16, 2024, <https://github.com/VikParuchuri/surya>.
51. "Inside Marker: A Guided Source Code Tour for an AI-Powered PDF Layout Detection Engine," accessed December 15, 2024, <https://journal.hexmos.com/marker-pdf-document-ai/>.
52. J. Berkenbilt, "Qpdf: A Content-Preserving PDF Document Transformer," accessed December 16, 2024, <https://github.com/qpdf/qpdf>.
53. "pypdfium," accessed December 16, 2024, <https://github.com/YinlinHu/pypdfium>.
54. "EasyOCR," accessed December 16, 2024, <https://github.com/JaideADAI/EasyOCR>.
55. "PyMuPDF," accessed December 10, 2024, <https://github.com/pymupdf/PyMuPDF>.
56. "Tesseract," accessed December 16, 2024, <https://github.com/tesseract-ocr/tesseract>.
57. R. Smith, "An Overview of the Tesseract OCR Engine," in *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, vol. 2 (Institute of Electrical and Electronics Engineers, 2007), 629–633.
58. "OpenCV," accessed December 10, 2024, <https://github.com/opencv/opencv-python>.
59. J. Canny, "A Computational Approach to Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence* 6 (1986): 679–698.
60. "Pillow," accessed December 10, 2024, <https://github.com/python-pillow/Pillow?tab=readme-ov-file>.
61. "Beyond Text," accessed December 11, 2024, [https://matextract.pub/content/beyond\\_text/beyond\\_images.html](https://matextract.pub/content/beyond_text/beyond_images.html).
62. KNIME, "Write Variables," accessed June 27, 2023, <https://hub.knime.com/vernalis/extensions/com.vernalis.knime.feature/latest/com.vernalis.knime.flowvar.nodes.io.write.WriteVariablesNodeFactory>.
63. KNIME, "Read Variables," accessed June 4, 2023, <https://hub.knime.com/vernalis/extensions/com.vernalis.knime.feature/latest/com.vernalis.knime.flowvar.nodes.io.read.ReadVariablesNodeFactory>.
64. "LiteLLM," accessed December 19, 2024, <https://github.com/BerriAI/litellm>.
65. T. Saddler, "ToxPipe," accessed December 19, 2024, <https://toxpipe.niehs.nih.gov>.
66. Y. Hu, Q. Chen, J. Du, et al., "Improving Large Language Models for Clinical Named Entity Recognition via Prompt Engineering," *Journal of the American Medical Informatics Association* 31 (2024): 1812–1820.
67. T. Pandas, "Pandas Development Team," 2020 pandas-dev/pandas, <https://pandas.pydata.org/>.
68. W. Xuying, Z. Jiangbo, Z. Yuping, et al., "Effect of Astragaloside IV on the General and Peripartum Reproductive Toxicity in Sprague-Dawley Rats," *International Journal of Toxicology* 29 (2010): 505–516.
69. F. F. Heuschmid, S. Schneider, P. Schuster, B. Lauer, and B. Ravenzwaay, "Developmental Toxicity of Polyethylene Glycol-g-Polyvinyl Alcohol Grafted Copolymer in Rats and Rabbits," *Food and Chemical Toxicology* 51 (2013): S14–S23.
70. N. J. Althali, A. M. Hassan, and M. A. Abdel-Wahhab, "Effect of Grape Seed Extract on Maternal Toxicity and in Utero Development in Mice Treated With Zearalenone," *Environmental Science and Pollution Research* 26 (2019): 5990–5999.
71. "Playground," accessed December 24 2024, <https://platform.openai.com/playground/chat>.
72. G. Gartlehner, L. Kahwati, R. Hilscher, et al., "Data Extraction for Evidence Synthesis Using a Large Language Model: A Proof-Of-Concept Study," *Research Synthesis Methods* 15 (2024): 576–589.
73. B. Alawaji, M. Hakami, and B. Alshemaimri, "Evaluating Generative Language Models With Prompt Engineering for Categorizing User Stories to its Sector Domains," in *2024 IEEE 9th International Conference for Convergence in Technology (I2CT)* (Institute of Electrical and Electronics Engineers, 2024), 1–8.
74. L. Reynolds and K. McDonnell, "Prompt Programming for Large Language Models: Beyond the Few-Shot Paradigm," in *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems* (Association for Computing Machinery, 2021), 1–7.

## Supporting Information

Additional supporting information can be found online in the Supporting Information section. **Data S1:** Supporting Information. **Data S2:** File S1. **Data S3:** File S2.